

PHASE 2 — Dashboard & Employee Management

PRSECURITY HRMS

IMPORTANT RULES

Output every file 100% completely. Never use '...' or 'rest remains same'.

After each file write "--- FILE COMPLETE ---" then start the next file immediately.

If about to hit output limit, finish current file and write "--- PAUSED: Type CONTINUE ---"

This is Phase 2 of 8. Phase 1 already built: Auth, DB schema skeleton, Sidebar, Topbar, Login, all route placeholders.

DO NOT rebuild Phase 1 files. Only add NEW files or EXTEND existing ones where specified.

CONTEXT

Project: PRSECURITY CONSULTANCY & SERVICES internal HRMS Stack:

React+Vite+TS+Tailwind+shadcn/ui frontend | Node+Express+TS+Prisma+PostgreSQL backend Roles: ADMIN, HR, EMPLOYEE, INTERN All amounts: INR ₹ | Format dates: DD MMM YYYY

STEP 1 — Update Prisma Schema

Expand the User and SalaryStructure models (already exist) — no changes needed. The placeholder models are fine for now.

STEP 2 — Shared UI Components

Build these reusable components used across all pages:

FILE: frontend/src/components/shared/StatCard.tsx

Props: `{ title: string, value: string | number, icon: LucideIcon, trend?: { value: number, label: string }, colorVariant?: 'primary' | 'success' | 'warning' | 'danger' }` Design: white card, icon in colored circle top-right, large value, small title below, optional green/red trend arrow at bottom.

FILE: frontend/src/components/shared/PageHeader.tsx

Props: `{ title: string, subtitle?: string, breadcrumbs?: { label: string, href?: string }[], actions?: ReactNode }` Design: title + subtitle left, breadcrumb below title, action buttons right.

FILE: frontend/src/components/shared/DataTable.tsx

Props: `{ columns: ColumnDef[], data: any[], isLoading?: bool, pagination?: { page, limit, total, onPageChange }, onSearch?: fn, searchPlaceholder?: string, filters?: ReactNode, onExport?: fn }` Features: sortable columns, search input, pagination controls, export CSV button, skeleton rows when loading, empty state when no data.

FILE: frontend/src/components/shared/StatusBadge.tsx

Props: `{ status: string }` Maps status strings to colours: ACTIVE/APPROVED/PRESENT/DONE=green, PENDING/IN_PROGRESS=yellow, INACTIVE/REJECTED/ABSENT=red, ON_LEAVE/CANCELLED=blue, TERMINATED=grey.

FILE: frontend/src/components/shared/Avatar.tsx

Props: `{ src?: string, name: string, size?: 'sm' | 'md' | 'lg' }` Shows image if src provided, else initials (first+last name chars) on navy background.

FILE: frontend/src/components/shared/ConfirmModal.tsx

Props: `{ isOpen, onClose, onConfirm, title, description, confirmLabel?, confirmVariant? }` shadcn/ui Dialog with cancel + confirm buttons.

FILE: frontend/src/components/shared/EmptyState.tsx

Props: `{ title, description, action?: { label, onClick } }` Centered icon + title + description + optional button.

FILE: frontend/src/components/shared/FileUpload.tsx

Props: `{ onFileSelect, accept?, maxSizeMB?, label? }` Drag-and-drop zone with click fallback. Shows file name after selection. Validates type and size.

STEP 3 — Employee Management Backend

FILE: backend/src/routes/employee.routes.ts

```
□  
  
GET  /api/employees      → getAllEmployees (HR/Admin)  
POST /api/employees      → createEmployee (HR/Admin)  
GET  /api/employees/:id  → getEmployee (authenticated)  
PUT   /api/employees/:id  → updateEmployee (HR/Admin or own profile)  
DELETE /api/employees/:id → deactivateEmployee (Admin only)  
POST  /api/employees/:id/documents → uploadDocument (Multer single file)  
GET   /api/employees/:id/documents → getDocuments  
GET   /api/employees/stats → getEmployeeStats (HR/Admin)
```

FILE: backend/src/services/employee.service.ts

`getAllEmployees(query):`

Accept query params: page (default 1), limit (default 20), search (name or employeeId), role, status

Return paginated list with salaryStructure included

Exclude passwordHash, refreshToken, passwordResetToken from all responses

`createEmployee(data):`

Auto-generate employeeId: query max existing PRS-XXX number, increment by 1, format as PRS-XXX (3 digits)

Hash a temporary password (format: Prs@{employeeId} e.g. Prs@PRS-002)

Create User + SalaryStructure in a Prisma transaction

Return created user

`getEmployee(id):`

Return full profile with salaryStructure

If requester is EMPLOYEE/INTERN, only return if id matches their own

`updateEmployee(id, data):`

HR/Admin can update everything including salary

Employee can only update: phone, address, emergencyContactName, emergencyContactPhone, bankAccountNumber, bankIFSC, bankName, panNumber

```
deactivateEmployee(id):
```

Set status = INACTIVE (soft delete, never hard delete)

```
uploadDocument(userId, file):
```

Multer saves to uploads/documents/{userId}/

Return file path

```
getEmployeeStats():
```

Return: totalActive, totalInactive, totalInterns, totalHR, totalAdmin, newThisMonth

FILE: backend/src/controllers/employee.controller.ts

Express controllers calling employee.service for all routes above.

FILE: backend/src/middleware/upload.middleware.ts

Multer configuration:

diskStorage with destination: uploads/{subfolder}/{userId}/

filename: {timestamp}_{originalname}

fileFilter: allow only pdf, jpg, jpeg, png

limits: 5MB

STEP 4 — Employee Management Frontend

FILE: frontend/src/api/employee.api.ts

Functions:

```
getAllEmployees(params) → GET /employees with query params
```

```
createEmployee(data) → POST /employees
```

```
getEmployee(id) → GET /employees/:id
```

```
updateEmployee(id, data) → PUT /employees/:id
```

```
deactivateEmployee(id) → DELETE /employees/:id
```

```
uploadDocument(id, file) → POST /employees/:id/documents (FormData)
```

```
getDocuments(id) → GET /employees/:id/documents
```

```
getEmployeeStats() → GET /employees/stats
```

FILE: frontend/src/pages/employees/EmployeesPage.tsx

Full employee list page:

PageHeader: "Employees" + "Add Employee" button (HR/Admin only)

StatCards row: Total Active, Total Interns, New This Month, On Leave

DataTable with columns:

Avatar + Full Name (clickable → /employees/:id)

Employee ID

Designation

Role badge

Status badge

Joining Date

Actions: View, Edit, Deactivate (Admin only)

Search by name or employee ID

Filter dropdown: Role, Status

Export CSV

Confirm modal for deactivation

FILE: frontend/src/pages/employees/NewEmployeePage.tsx

Multi-step form (5 steps) with progress indicator at top:

Step 1 — Personal Info:

First Name*, Last Name*, Date of Birth, Phone, Address, Emergency Contact Name, Emergency Contact Phone

Step 2 — Employment Info:

Designation*, Role* (dropdown: HR/EMPLOYEE/INTERN), Employment Type*, Date of Joining*

Step 3 — Salary Structure:

Basic Salary (₹)*, HRA (₹), Travel Allowance (₹), Medical Allowance (₹), Other Allowances (₹)

PF Employee % (default 12), PF Employer % (default 12)

ESI Employee % (default 0.75), ESI Employer % (default 3.25)

TDS % (default 0)

Live preview: shows calculated Gross and Net Salary as user types

Step 4 — Bank Details:

Bank Name, Account Number, IFSC Code, PAN Number

Step 5 — Review & Submit:

Summary of all entered data

Submit button

Show temporary password that will be assigned (Prs@{employeeId})

Navigation: Previous / Next buttons, validation on each step before proceeding. On success: toast + redirect to /employees/:id

FILE: frontend/src/pages/employees/EmployeeDetailPage.tsx

Full profile page with tabs:

Tab 1 — Overview:

Profile picture (with upload button), name, employee ID, designation, role badge, status badge

Info grid: Email, Phone, DOB, Joining Date, Employment Type, Address

Emergency contact section

Bank details section (masked account number)

Edit Profile button → opens edit modal (same fields as step 1+4 of create form)

Tab 2 — Salary:

Salary structure display (earnings table + deduction percentages)

Edit Salary button (HR/Admin only) → opens salary edit modal

Tab 3 — Documents:

List of uploaded documents with download links

Upload new document button → FileUpload component

Remaining tabs (Attendance, Leave, Payslips, Tasks) → show EmptyState with "Available in next update" for now. Will be filled in later phases.

FILE: frontend/src/pages/employees/EditEmployeeModal.tsx

Reusable modal for editing employee fields. Two variants based on role:

HR/Admin: all fields editable

Employee: only personal + bank fields

STEP 5 — Dashboards Backend

FILE: backend/src/routes/dashboard.routes.ts

```
GET /api/dashboard/admin → Admin/HR dashboard stats
GET /api/dashboard/employee → Employee/Intern dashboard stats
```

FILE: backend/src/services/dashboard.service.ts

getAdminDashboardStats(userId): Return:

```
json
{
  "totalEmployees": 0,
  "presentToday": 0,
  "onLeaveToday": 0,
  "openTickets": 0,
  "activeProjects": 0,
  "pendingLeaveRequests": 0,
  "pendingExpenseClaims": 0,
  "recentEmployees": [...last 5 joined],
  "headcountByRole": [{ "role": "EMPLOYEE", "count": 0 }]
}
```

(Most counts will be 0 initially — will be updated as modules are added)

getEmployeeDashboardStats(userId): Return:

```
json
{
  "todayAttendance": null,
  "leaveBalance": [],
  "taskStats": { "todo": 0, "inProgress": 0, "done": 0 },
  "latestPayslip": null,
  "announcements": []
}
```

FILE: backend/src/controllers/dashboard.controller.ts

STEP 6 — Dashboard Frontend

FILE: frontend/src/pages/dashboard/AdminDashboardPage.tsx

Admin/HR dashboard:

Greeting: "Good Morning/Afternoon/Evening, [Name]" based on time

Stats row (6 StatCards): Total Employees, Present Today, On Leave Today, Open Tickets, Active Projects,

Pending Approvals

Row 2:

Left (60%): Bar chart (Recharts) — Headcount by Role

Right (40%): Recent Employees list (avatar + name + joining date)

Row 3:

Left: Pending Leave Requests table (name, type, dates, approve/reject quick buttons)

Right: Quick Actions card (Add Employee, Post Announcement, Run Payroll, View Reports — each a button linking to that page)

FILE: frontend/src/pages/dashboard/EmployeeDashboardPage.tsx

Employee/Intern dashboard:

Greeting with name + current date

Stats row (4 StatCards): Today's Status (Present/Absent), Leave Balance (total remaining), My Tasks (open), Latest Payslip Month

Row 2:

Left: Punch In/Out widget — large button showing current time, status badge, today's punch times. (Attendance API will be connected in Phase 3 — for now show placeholder state)

Right: Leave Balance cards (one mini card per leave type — will be populated in Phase 4)

Row 3: Announcements feed (empty state for now)

FILE: frontend/src/pages/dashboard/DashboardPage.tsx

Router component: reads role from authStore, renders AdminDashboardPage or EmployeeDashboardPage accordingly.

STEP 7 — Notifications Backend (Basic)

FILE: backend/src/services/notification.service.ts

Functions:

`createNotification(userId, title, message, type, link?)` → creates DB record + emits socket event to that user

`getNotifications(userId, page, limit)` → paginated list

`markAsRead(notificationId, userId)` → update isRead

`markAllAsRead(userId)` → update all

FILE: backend/src/routes/notification.routes.ts

```

❏
GET /api/notifications    → getNotifications (authenticated)
PUT /api/notifications/:id/read → markAsRead
PUT /api/notifications/read-all → markAllAsRead
```

FILE: backend/src/controllers/notification.controller.ts

STEP 8 — Notifications Frontend

FILE: frontend/src/api/notification.api.ts

Functions for all notification endpoints.

FILE: frontend/src/components/layout/NotificationDropdown.tsx

Bell icon component (goes in Topbar):

Badge with unread count (red circle)

Dropdown panel: list of notifications (title, message, time ago)

Each notification clickable → navigate to link if provided, mark as read

"Mark all as read" button at top

Fetches on mount + polls every 30 seconds

Uses Socket.io to receive real-time new notifications

Update frontend/src/components/layout/Topbar.tsx

Replace placeholder notification bell with NotificationDropdown component.

AFTER ALL FILES ARE COMPLETE

Write this exactly: "  PHASE 2 COMPLETE — All shared components, Employee Management (list, create, detail, edit), Role-based Dashboards, and Notifications system are ready. Proceed to Phase 3 for Attendance Management."